

```
//velocity
if (!isClimbing && !sittingInJumper)
{
    hor = Input.GetAxis("Horizontal");
    vert = rb.velocity.y;
}

rb.velocity = new Vector2(hor * movingSpeed, vert);
```

Передвижение игрока выполняется через Rigidbody2D

```
//jump
if (Input.GetKeyDown(KeyCode.Space) && (onGround || isClimbing || sittingInJumper))
{
    //jump
    if (!jumpSound.isPlaying)
    {
        jumpSound.Play();
    }
    transform.parent = null;
    sittingInJumper = false;
    rb.isKinematic = false;
    isClimbing = false;
    animator.SetBool("climbingStay", false);
    animator.SetBool("climbing", false);
    jumpingSpeed = movingSpeed;
    rb.AddForce(Vector2.up * jumpImpulse, ForceMode2D.Impulse);
}
```

прыжок выполняется по нажатию пробела и только тогда, когда игрок стоит на земле. При этом включается анимация прыжка, спрайт игрока отключается, и включается соответствующий звук

```
//animation
if (hor != 0 && !isClimbing && !sittingInJumper)
{
    if (!walkSound.isPlaying)
    {
        walkSound.Play();
    }

    animator.SetBool("walk", true);
}
else
{
    walkSound.Stop();
    animator.SetBool("walk", false);
}
```

если игрок стоит на месте – анимация ходьбы переходит в анимацию стойки, звук шагов выключается, при начале ходьбы анимация включается и звук проигрывается

```

    if (isClimbing)
    {
        vert = Input.GetAxis("Vertical");
        if (transform.position.y <= upStringPoint.position.y && transform.position.y >= downStringPoint.position.y)
        {
            rb.velocity = new Vector2(0, vert * climbingSpeed);
        }
        else
        {
            if (transform.position.y > upStringPoint.position.y)
            {
                transform.position = new Vector3(transform.position.x, upStringPoint.position.y, transform.position.z);
            }
            if (transform.position.y < downStringPoint.position.y)
            {
                transform.position = new Vector3(transform.position.x, downStringPoint.position.y, transform.position.z);
            }
            rb.velocity = new Vector2(0, 0);
        }

        if (vert == 0)
        {
            animator.SetBool("climbingStay", true);
        }
        else
        {
            animator.SetBool("climbingStay", false);
        }
    }
}

```

если игрок висит на верёвке он управляет клавишами w и s, при этом если он движется, включается анимация взбирания, если остановился – выключается.

Если игрок попадает в триггер, который может его убить, то сцена перезапускается, а у игрока включается анимация смерти

```

    if (collision.tag == "fire")
    {
        //death
        GetComponent<Rigidbody2D>().isKinematic = true;
        movingSpeed = 0;
        animator.enabled = false;
        GetComponent<SpriteRenderer>().enabled = false;
        deathAnima.SetActive(true);
        deathAnima.GetComponent<Animator>().SetBool("gas", true);
        StartCoroutine(WaitBeforeRestart());
    }
}

```

Если игрок подходит к предмету, который можно взять, то тот перемещается в свободный слот (их всего 2)

```

public void PushImageToInventory(GameObject obj, Sprite slotSprite)
{
    freeSlot++;
    itemsNameInSlot[freeSlot] = obj.GetComponent<InventoryItem>().numberOfItem;
    SpawnInventory();
    inventoryObjects.Add(obj);
    if (inventoryObjects.Count > 2)
    {
        inventoryObjects.Remove(inventoryObjects[0]);
        inventoryObjects.Remove(inventoryObjects[0]);
    }
    StartCoroutine(MoveToInventory(obj, slots[freeSlot].gameObject, slotSprite));
}

```

```

Vector2 destination;
IEnumerator MoveToInventory(GameObject obj, GameObject slot, Sprite slotSprite)
{
    destination = (slot.transform.position - obj.transform.position).normalized;
    while (true)
    {
        if (Vector2.Distance(obj.transform.position, slot.transform.position) >= 1f)
        {
            obj.transform.Translate(destination * Time.deltaTime * movingToInventorySpeed);
            yield return new WaitForSeconds(0.02f);
        }
        else
        {
            slot.GetComponent<Image>().sprite = slotSprite;
            obj.SetActive(false);
            StopCoroutine(MoveToInventory(obj, slot, slotSprite));
            time = 0;
            StartCoroutine(WaitBeforeDeSpawn());
            break;
        }
    }
}

```

Когда предмет достиг ячейки, он выключается, а спрайт слота инвентаря заменяется на нужный.

```

public void SpawnInventory()
{
    StopCoroutine(Despawn());
    bool flag = false;
    foreach (var item in itemsNameInSlot)
    {
        if (item != -1)
        {
            flag = true;
        }
    }
    inventoryGo.SetActive(true);
    foreach (var slot in slots)
    {
        Color newColor_ = Color.white;
        newColor_.a = 0;
        slot.color = newColor_;
    }
    time = 0;
    if (!flag && !alreadyRedAnima)
    {
        alreadyRedAnima = true;
        foreach (var slot in slots)
        {
            slot.color = Color.red;
        }
        StartCoroutine(RedAnima());
    }
    if (flag)
    {
        StartCoroutine(Spawn());
    }
}

```

Когда игрок подходит к объекту, для которого нужен инвентарь, тот плавно включается,

Если предметов в инвентаре нет, то он мигает красным

```

IEnumerator RedAnima()
{
    while (numberOfRedAnimations != amountOfRedAnimation)
    {
        print("redAnima");*/

        if (slots[0].color.a < 1 && numberOfRedAnimations % 2 == 1)
        {
            print("redAnima+");*/
            time += Time.deltaTime;
            newColor = slots[0].color;
            newColor.a = (time / redTime) * 1;
            if (newColor.a >= 1)
            {
                newColor.a = 1;
            }

            foreach (var slot in slots)
            {
                slot.color = newColor;
                print("color" + slots[0].color.a);
            }
        }
        if (slots[0].color.a > 0 && numberOfRedAnimations % 2 == 0)
        {
            print("redAnima-");*/
            time += Time.deltaTime;
            newColor = slots[0].color;
            newColor.a = 1 - (time / redTime) * 1;
            if (newColor.a <= -0)
            {
                newColor.a = 0;
            }
            foreach (var slot in slots)
            {
                slot.color = newColor;
            }
        }

        if (slots[0].color.a == 1 || slots[0].color.a == 0)
        {
            time = 0;
            numberOfRedAnimations += 1;
        }

        yield return new WaitForSeconds(0.02f);
    }
    numberOfRedAnimations = 0;
    time = 0;
    foreach (var slot in slots)
    {
        slot.color = Color.white;
    }
    alreadyRedAnima = false;
    StopCoroutine(RedAnima());
}

```

```

Color newColor;
IEnumerator Spawn()
{
    while (slots[0].color.a < 1)
    {
        time += Time.deltaTime;
        newColor = slots[0].color;
        newColor.a = (time / spawnTime) * 1;
        foreach (var slot in slots)
        {
            slot.color = newColor;
        }

        yield return new WaitForEndOfFrame();
    }
}

```

При выходе игрока из триггера выключается инвентарь

```

IEnumerator Despawn()
{
    while (slots[0].color.a > 0)
    {
        time += Time.deltaTime;
        newColor = slots[0].color;
        newColor.a -= (time / spawnTime) * 1;
        foreach (var slot in slots)
        {
            slot.color = newColor;
        }

        yield return new WaitForEndOfFrame();
    }
}

```

Объекты в начале уровня передвигаются с помощью мышки, при попадании в триггер нужного слота они «прицепляются», а если игрок отпустил мышку, а при этом объект не находится в триггере, то объект перемещается в стартовую позицию

```

5
6 void IBeginDragHandler.OnBeginDrag(PointerEventData eventData)
7 {
8     offset = -transform.position + Camera.main.ScreenToWorldPoint(Input.mousePosition);
9 }
10
11 void IDragHandler.OnDrag(PointerEventData eventData)
12 {
13     Vector2 currentMousePosition = Camera.main.ScreenToWorldPoint(Input.mousePosition);
14     Vector2 diff = currentMousePosition - offset;
15     transform.position = new Vector3(diff.x, diff.y, 0);
16     transform.localPosition = new Vector3(transform.localPosition.x, transform.localPosition.y, 0);
17 }
18
19 void IEndDragHandler.OnEndDrag(PointerEventData eventData)
20 {
21     if (inCol)
22     {
23         transform.position = windowPos;
24         GetComponent<Image>().raycastTarget = false;
25     }
26     else
27     {
28         transform.position = startPos;
29     }
30 }

```

При попадании игрока в чекпоинт камера передвигается на следующую точку

```

public IEnumerator Move(Transform point)
{
    while (Vector2.Distance(transform.position, point.transform.position) >= 0.1f)
    {
        transform.Translate((point.transform.position - transform.position) * Time.deltaTime * movingSpeed);

        yield return new WaitForEndOfFrame();
    }
    if (pointNumb <= 2)
    {
        levelScripts[pointNumb].SetActive(true);
    }
    else
    {
        playerMover.enabled = true;
    }
}

```

При этом включается окно с цепочками взаимодействия

При попадании игрока в триггер звёздочки окно с ними появляется так же как и инвентарь, при этом звёздочка летит в свободный слот, при достижении которого у звёздочек включается нужная анимация

```

Vector2 destinationForStar;
IEnumerator MoveStar(Transform point, GameObject obj, int starPointNumb)
{
    destinationForStar = (point.position - obj.transform.position).normalized;
    while (Vector2.Distance(obj.transform.position, point.position) >= 1f)
    {
        if (Vector2.Distance(obj.transform.position, point.position) >= 1f)
        {
            obj.transform.Translate(destinationForStar * Time.deltaTime * movingStarSpeed);
            yield return new WaitForSeconds(0.02f);
        }
        else
        {
            obj.SetActive(false);
            images[starPointNumb].sprite = filledStar;
            StopCoroutine(MoveStar(point, obj, starPointNumb));
        }
    }
    obj.SetActive(false);
    images[starPointNumb].GetComponent<Animator>().SetBool("fiilling", true);
}

```

Когда игрок зашёл в чекпоинт и перешёл на новый стейдж, звёздочки снимаются, им назначается начальный спрайт

```

public void SetToDefault()
{
    foreach (var image in images)
    {
        starNumb = 0;
        image.GetComponent<Animator>().SetBool("fiilling", false);
    }
    StartCoroutine(WaitBeforeDespawn());
}

```